

Module 1: Introduction to Web Development

Learning Objectives

- Understand the fundamentals of web development.
- Learn basic HTML structure and syntax.
- Learn basic CSS styling and layout.
- Learn basic JavaScript interactivity.

Module Outline

1. **Introduction to the Internet and the Web**
2. **HTML (HyperText Markup Language)**
 - Basic HTML Structure
 - Tags, Elements, and Attributes
3. **CSS (Cascading Style Sheets)**
 - CSS Syntax and Selectors
 - Colors, Fonts, and Text Styles
 - Box Model and Layout
 - Responsive Design Principles
4. **JavaScript**
 - Introduction to JavaScript
 - Variables, Data Types, and Operators
 - DOM Manipulation
 - Basic Event Handling

Contents

1. Introduction to the Internet and the Web

The Internet is a vast, global network connecting millions of computers, enabling them to communicate and share information. Think of it as the worldwide infrastructure – the roads and highways. The World Wide Web (WWW), often used interchangeably with the Internet, WWW is a service on the Internet. It's a collection of interconnected documents and other web resources, linked by hyperlinks and URLs, accessible via web browsers. The Web uses the HTTP protocol to transmit data.

The web has undergone a remarkable transformation, evolving from a read-only medium to an interactive and increasingly decentralized landscape. This evolution is often categorized into three distinct phases: Web 1.0, Web 2.0, and the emerging Web 3.0.

- Web 1.0: The Static Web

The initial iteration of the World Wide Web, commonly referred to as Web 1.0, was primarily characterized by static content. Websites during this era were essentially digital brochures or online encyclopedias, offering information with very limited to no interactive elements. Users were largely passive consumers of content, with little opportunity to contribute or engage directly with the platforms they visited. The focus was on presenting information, and the architecture was predominantly one-way.

- Web 2.0: The Social and Interactive Web

The advent of Web 2.0 marked a significant shift towards a more interactive and user-centric internet. This phase, which encompasses the web as most people experience it today, empowers users to not only consume content but also to create, share, and collaborate. Platforms like social media networks, blogs, wikis, and video-sharing sites are hallmarks of Web 2.0. This era is defined by user-generated content, social connectivity, and a more dynamic, two-way flow of information, though often facilitated by centralized platforms that control user data.

- Web 3.0: The Decentralized Web

The current and developing phase, Web 3.0, introduces a fundamentally different architectural concept. Unlike its predecessors, Web 3.0 is built upon the principles of decentralization. Leveraging technologies such as blockchain, cryptocurrencies, and artificial intelligence, Web 3.0 aims to create a more transparent, secure, and user-controlled internet. In this model, there is no single point of control or a central provider; instead, data and applications are distributed across a network. This shift promises greater user privacy, data ownership, and resistance to censorship, heralding a new era of online interaction and a more equitable digital world.

2. HTML (HyperText Markup Language)

HTML is the standard markup language used to create the structure and content of web pages. It tells web browsers how to display text, images, and other media.

- Basic HTML Structure

Every HTML element is defined by a starting tag, some contents, and an ending tag, with the format as the following:

```
<tagname> content </tagname>
```

Tagname can be replaced with whatever kind of HTML element the programmers would like to use to display the content.

```
<!DOCTYPE html>
<html>
<head>
  <title>Page Title</title>
</head>
<body>
  <h1>This is a Heading</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

The overall structure of HTML code consists of a `<head>` and a `<body>`. The `<head>` part of HTML contains machine-readable information about the document, such as the title, scripts, meta, and style sheets, which will be explained further in the CSS and JS section. This part of HTML primarily holds information for machines to process, not to be displayed into web browsers. On the other hand, the `<body>` part of HTML contains all the information and contents of an HTML document. This part of HTML might contain elements such as headings, paragraphs, images, etc.

- Tags, Elements, and Attributes

Tags: These are the building blocks of HTML, enclosed in angle brackets (`< >`). Most tags come in pairs: an opening tag (`<p>`) and a closing tag (`</p>`).

Elements: An element consists of an opening tag, the content, and a closing tag (e.g., `<p>This is a paragraph.</p>`). Some elements are self-closing (e.g., `<img>`).

Attributes: Provide additional information about an HTML element. They are always included in the opening tag and come in name/value pairs (e.g., `<a href="https://www.google.com" />`).

Element	Tagname	Description
Headings	<code><h1> </h1></code> <code><h2> </h2></code> <code><h3> </h3></code> <code><h4> </h4></code> <code><h5> </h5></code> <code><h6> </h6></code>	Heading is used to display titles or subtitles on a webpage. Various types of headings are provided by HTML, sorted with <code><h1></code> being the largest displayed.
Paragraphs	<code><p> </p></code>	Paragraph element is used to display a block of text content.
Formatting	Bold: <code> </code> or <code> </code>	Formatting can be flexibly used by programmers to display the

	Italic: <i> </i> Underlined: <u> </u> Small Text: <small> </small> Highlight: <mark> </mark> Code: <code> </code>	content on a webpage.
Quotations	<q> </q>	Quotations automatically add quotation marks to the content it contains.
Comments	<!-- -->	Comments used to write a comment or note in the code file without displaying it into the web page.
Links	 	Links are used to redirect a web page into a certain URL when clicked.
Images		Images are used to display images using a certain source of images (can be a URL or a path of the image).
Tables	<pre><table> <tr> <th> </th> <th> </th> </tr> <tr> <td></td> <td></td> </tr> </table></pre>	Table elements are used to display tables in the web page. <tr> tags are the row of the table. The first row of the table is usually inserted by <th> tags which are the table heading. On the other hand, the rest of the rows are inserted using <td> tags which are the table data.

Lists	Unordered List: <pre> </pre> Ordered List: <pre> </pre>	There are two types of provided listing methods in HTML. The unordered lists, or bulleted lists, are used to list contents without considering the order. On the other hand, ordered lists are usually called numbered lists where the order of the list is necessary.
Divisions	<pre><div> </div> </pre>	The <code><div></code> tags are used for block type of styling while the <code></code> tags are used for in-line type of styling.
Buttons	<pre><button> </button></pre>	Buttons are used to display buttons that are interactable to users.
Forms	<pre><form> </form></pre>	HTML forms provide a structured way to gather information from users. By using the <code><form></code> element as a container for various input controls (such as <code><input></code>, <code><textarea></code>, and <code><select></code>), you can collect diverse data types. This collection of data is then typically submitted as a single package to a web server for handling, such as logging in, registering, or placing an order.
Input	<pre><input type="text" ></pre>	<code><input></code> elements are the primary way to gather user data within a form. These versatile elements function as interactive controls, and their specific role is defined by the crucial <code>type</code> attribute. This

		allows you to easily create different kinds of inputs, such as single-line text boxes, password fields that obscure text, clickable radio buttons for single selections, checkboxes for multiple selections, and specialized fields for entering emails, numbers, or dates.
Etc. (HTML 5)		Refer to https://www.geeksforgeeks.org/html5-new-tags/

3. CSS (Cascading Style Sheets)

CSS is a stylesheet language used to describe the presentation (look and formatting) of a document written in HTML. It controls colors, fonts, layout, and more.

- CSS Syntax and Selectors

Every CSS rule consists of a selector and a declaration block. Selector points to the HTML elements that the CSS is applied to. On the other hand, the declaration block contains the statements of style that the programmers would like to apply to the selector. Each declaration block might consists of several declarations, while every CSS declaration has the syntax of,

```
(selector) {
  (property) : (value);
  (property) : (value);
}
```

for example:

```
h1 {
  color: blue;
  font-weight: bold;
}
```

Property is the type of style used by the programmer, while the value, as how it is named, is the value of the property itself. In the example above, the programmers would like to set, for all h1 elements, the value of blue to the color property and the value of bold to the font-weight property.

There are three ways of approach in order to use CSS, the **inline**, **internal**, and **external**.

1. **Inline CSS** is a way of defining the styling of an HTML element by adding CSS rules directly to the element's tag using 'style' attribute. This kind of approach is effective when programmers would like to affect only a single or a group of elements.

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Web Development Roadmap</title>
</head>
<body>
  <h1 style='color:blue; font-weight:bold'>GDGoC ITB</h1>
</body>
</html>
```

2. **Internal CSS** approach is more structured than the inline approach. It used the <style> tagname inside the <head> to define all the styles that would be used. This approach is effective when the programmers would like to affect multiple elements within the same HTML document.

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Web Development Roadmap</title>
  <style>
    h1 {
      color:blue;
```



```

    font-weight: bold;
}
</style>
</head>
<body>
  <h1>GDGoC ITB</h1>
</body>
</html>

```

3. **External CSS** approach is the most structured compared to the previous two. It uses a separate .css file and <link> tagname to import the specific .css file to be used inside an HTML document. This approach is effective when programmers would like to reuse the style declarations and use it to affect HTML elements across multiple HTML documents.

index.html	style.css
<pre> <!DOCTYPE html> <html> <head> <title>Web Development Roadmap</title> <link rel="stylesheet" type="text/css" href="style.css"> </head> <body> <h1>GDGoC ITB</h1> </body> </html> </pre>	<pre> h1 { color:blue; font-weight: bold; } </pre>

To make CSS not to affect all elements of the same tagname, programmers would like to use 'class' attribute to set which elements to be styled and which elements not to. Thus, CSS will look into the specific 'class' to style the HTML element.

index.html	style.css
<pre> <!DOCTYPE html> <html> </pre>	<pre> .red-bold { color:red; } </pre>

<pre><head> <title>Web Development Roadmap</title> <link rel="stylesheet" type="text/css" href="style.css"> </head> <body> <h1 class='red-bold'>GDGoC</h1> <h1 class='blue-bold'>ITB</h1> </body> </html></pre>	<pre>font-weight: bold; } .blue-bold{ color: blue; font-weight: bold; }</pre>
---	--

- CSS Selector Priority

When multiple CSS rules target the same HTML element, the browser needs a way to decide which rule "wins" and gets applied. This is where **CSS selector priority**, or **specificity**, comes in. Think of it as a scoring system for your selectors.

Here's a brief breakdown of how it works, from highest to lowest priority:

1. **!important Rule:** If you add `!important` to a style declaration (e.g., `color: red !important;`), it will override almost any other declaration. It's a powerful tool but should be used sparingly as it can make debugging difficult.
2. **Inline Styles:** Styles applied directly to an HTML element using the `style` attribute (e.g., `<p style="color: blue;">`) have the next highest priority.
3. **ID Selectors:** Selectors using an ID (e.g., `#main-content`) are very specific and carry a high weight.
4. **Classes, Attribute Selectors, and Pseudo-classes:** This group comes next. This includes:
 - Class selectors (e.g., `.my-class`)
 - Attribute selectors (e.g., `[type="text"]`)
 - Pseudo-classes (e.g., `:hover`, `:focus`)
5. **Element and Pseudo-element Selectors:** These have the lowest specificity. This includes:
 - Element/Type selectors (e.g., `p`, `h1`, `div`)
 - Pseudo-elements (e.g., `::before`, `::after`)

The Tie-Breaker: The Cascade

What happens if two rules target the same element and have the *exact same specificity score*? In this case, the **last rule defined** in your CSS code wins. This is part of the "Cascading" nature of Cascading Style Sheets (CSS).

In essence, the browser calculates a score for each selector based on how many IDs, classes, and elements it contains. The selector with the highest score gets its styles applied. If scores are tied, the last one read by the browser takes precedence.

- Responsive Design Principles

Responsive design ensures that web pages look good on all devices (desktops, tablets, and phones). Key principles include:

1. Fluid Grids: Using relative units (like percentages) for widths, rather than fixed units.
2. Flexible Images: Ensuring images scale appropriately.
3. Media Queries: Applying different CSS rules based on the device's characteristics (like screen width).

4. JavaScript

- Introduction to JavaScript

JavaScript is a programming language, commonly used in website development. JavaScript file has the format of .js and .jsx. A simple JavaScript scripting language to add functionality into a website can be stored in .js format. On the other hand, .jsx, which stands for 'JavaScript XML', is a mixture of JavaScript and HTML. However, .jsx files are commonly used when developing a website using a framework. Therefore, a further explanation regarding .jsx files will be discussed in the next topic. If HTML acts as the skeleton of a web page and CSS acts as the skin and looks of a web page, JavaScript acts as the muscle to make a web page responsive and functional.

JavaScript commonly utilizes the 'id' attribute of HTML elements. The 'id' attribute is the one used to differentiate between one and another HTML elements when searched by JavaScript.

<code>document.getElementById({idname}).{part} = {value}</code>
for example:
<code>document.getElementById("GDGoC-title").innerHTML = 'GDGoC ITB';</code>

Using this kind 'id' attribute, JavaScript can be used to modify the HTML elements however the programmer is pleased.

Just like another programming language, JavaScript can be used for variable declarations, function making, etc. There are two approaches in order to use JavaScript, internal and external. In the internal approach, the JavaScript codes are defined inside the HTML document.

index.html
<pre> <!DOCTYPE html> <html> <head> <title>Web Development Roadmap</title> </head> <body> <h1 id='GDGoC-title' onclick='handleClick()'>GDGoC ITB</h1> <script> function handleClick() { let GDGoCString = 'Google Developer Group on Campus'; document.getElementById('GDGoC-title').innerHTML = GDGoCString; } </script> </body> </html> </pre>

On the other hand, an external JavaScript code file is imported into the HTML document.

index.html	script.js
<pre> <!DOCTYPE html> <html> <head> <title>Web Development Roadmap</title> </head> <body> <h1 id='GDGoC-title' onclick='handleClick()'>GDGoC ITB</h1> <script src="script.js"></script> </body> </html> </pre>	<pre> function handleClick() { let GDGoCString = 'Google Developer Group on Campus'; document.getElementById('GDGoC-title').innerHTML = GDGoCString; } </pre>

- Variables, Data Types, and Operators

Variables: Used to store data values. In JavaScript, you can declare variables using `var`, `let`, or `const`.

Data Types: JavaScript has several data types, including:

- **String:** Textual data (e.g., `"Hello"`).
- **Number:** Numerical data (e.g., `100`, `3.14`).
- **Boolean:** Represents `true` or `false`.
- **Object:** A collection of key-value pairs.

Operators: Used to perform operations on variables and values (e.g., `+` for addition, `*` for multiplication, `===` for strict equality).

- DOM Manipulation

The **Document Object Model (DOM)** is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. JavaScript can access and manipulate the DOM to:

1. Find HTML elements.
2. Change HTML content and attributes.
3. Change CSS styles.
4. Add or remove HTML elements and attributes.

index.html	script.js
<pre><!DOCTYPE html> <html> <head> <title>Web Development Roadmap</title> </head> <body> <h1 id='GDGoC-title' onclick='handleClick()'>GDGoC</pre>	<pre>function handleClick() { let GDGoCString = 'Google Developer Group on Campus'; document.getElementById('GDGoC-title').innerHTML = GDGoCString; }</pre>

ITB</h1> <script src="script.js"></script> </body> </html>	
---	--

- Basic Event Handling

JavaScript can be used to handle various events of HTML elements. In the code above, JavaScript is used to handle click events on HTML elements with the 'id' of 'GDGoC-title'. There are several common events such as, scroll event (*onscroll*), change event (*onchange*), submit event (*onsubmit*), that can be used by the programmers to make a web page more intuitive.

index.html	script.js
<!DOCTYPE html> <html> <head> <title>Web Development Roadmap</title> </head> <body> <h1 id='GDGoC-title'>GDGoC ITB</h1> <script src="script.js"></script> </body> </html>	function handleClick() { let GDGoCString = 'Google Developer Group on Campus'; document.getElementById('GDGoC-title').innerHTML = GDGoCString; } document.getElementById('GDGoC-title').addEventListener('click', handleClick())

Learning Materials

Topic	Materials
HTML Basics	https://developer.mozilla.org/en-US/docs/Web/HTML
CSS Basics	https://developer.mozilla.org/en-US/docs/Web/CSS
JavaScript Basics	https://developer.mozilla.org/en-US/docs/Web/JavaScript

Assessment

1. Create a web page of a self-portfolio, consisting of a minimum three pages. Members can freely decide the content of the pages. However, there should be a page that is used as the homepage/ landing page.
2. In order to connect one page to another, the website should have a navigation bar (navbar). The navbar is interchangeable with a sidebar, as long as the functionality requirement is fulfilled.
3. There are no specifications regarding the styling of the web page. Members could freely style and design their own web page up to their creativity.
4. JavaScript implementation has to be done in the webpage. There should be at least 2 events handled by JavaScript.
5. The web page should be able to display at least an image, an audio, or a video.
6. The webpage should feature an interactive section with a text input field (e.g., a `<textarea>`). As the user types into this field, their text should be displayed in **real-time** within a designated output area on the same page.
7. The solution must utilize the three standard methods for CSS integration (inline, internal, and external) and the two standard methods for JavaScript integration (internal and external)